

HVK IBM Hardware Pilot: Supplementary Methodology

Sparsho Chakraborty and Siddhartha Patra

Abstract

This is a companion methodology document to the main manuscript, *Do the Quantum Parts Help? A Component-Wise Ablation of a Physics-Informed Hybrid Quantum-Classical Image Autoencoder*. It documents, in full, the real-hardware reconstruction pilot summarized there in Section V.F: exact circuit construction, local validation against the training-time simulator, IBM Quantum job identifiers, shot budgets, and quota accounting. We separate this material from the main text so that the paper stays focused on the result, while every implementation detail needed to reproduce or audit the hardware run is preserved here.

1 Why a Separate Document

The main manuscript already carries a compiled-circuit feasibility appendix (transpiled depth and gate counts on a Heron-class basis, no finite-shot execution). This pilot is a distinct, additional contribution — an actual finite-shot hardware execution, decoded end-to-end into reconstructed pixels using an already-trained checkpoint’s exact parameters, with no retraining. Because the methodology here is API/infrastructure detail (account setup, backend selection, job submission, quota bookkeeping) rather than a scientific claim in its own right, we keep it out of the main text and report it here in full for reproducibility and audit.

2 Reused Checkpoints

No model was retrained for this pilot. Two pre-existing artifacts were reused directly:

- **Monalisa (HVK1D)**. The “shared VQC baseline” checkpoint (`Main2/newHVK/results/ablation_study/1`) is the same run underlying the “Legacy baseline, signed energy” row of the main paper’s Hamiltonian-controls table (32.24 dB simulator PSNR). Its `model.pt` holds `feature_projection` ($\mathbb{R}^{46 \rightarrow 6}$), `position_projection` ($\mathbb{R}^{8 \rightarrow 6}$), the StronglyEntanglingLayers weight tensor ($2 \times 6 \times 3$), and the Heisenberg couplings J_x, J_y, J_z ; `decoder.pt` holds a three-layer MLP decoder ($35 \rightarrow 128 \rightarrow 256 \rightarrow 4096$, ReLU/ReLU/Sigmoid).
- **CIFAR-10 (HVK2D)**. Four fresh checkpoints were trained (200 steps, Adam, $\eta = 0.004$, on an NVIDIA GTX 1050) for four held-out CIFAR-10 images at native 32×32 resolution, using non-overlapping 8×8 patches (16 patches/image, chosen to match the Monalisa pilot’s patch count and keep the real-hardware circuit budget predictable) rather than the main paper’s overlapping-stride same-set protocol. This is a deliberate simplification for the hardware pilot and is *not* the paper’s held-out CIFAR-10 statistical suite (Section V.B of the main text); it is a separate, smaller-scale demonstration.

3 Circuit Reconstruction

3.1 HVK1D (Monalisa)

The trained forward pass (`Main/src/quantum/quantum_model.py`) applies, per patch: `AngleEmbedding(inputs)` (PennyLane default rotation, R_X per wire), R_Y (positional_angles) per wire, then two `StronglyEntanglingLayers` (per-wire $\text{Rot}(\phi, \theta, \omega)$ decomposed as $R_Z R_Y R_Z$, followed by a CNOT ring whose range increments by layer). We rebuilt this gate-for-gate in Qiskit rather than trusting a hand derivation: the script author’s manual translation was checked against PennyLane’s own tape output before use.

Because the trained circuit measures Z , X , ZZ , XX , and YY (27 observables total: $6+6+5+5+5$), and $ZZ_{i,i+1}/XX_{i,i+1}/YY_{i,i+1}$ are recoverable as joint parities of the *same* all- Z , all- X , and all- Y basis measurements that already give the single-qubit marginals, only **three** measurement circuits per patch are needed (not 27): an all- Z -basis circuit, an all- X -basis circuit (Hadamard before measurement), and an all- Y -basis circuit ($S^\dagger$ then Hadamard before measurement). For 16 Monalisa patches this is $16 \times 3 = 48$ circuits.

3.2 HVK2D (CIFAR-10)

The HVK2D grid circuit (`Main2/src/model.py`) is simpler: two layers of (CNOT over the fixed grid edge set $\{(0,1), (1,2), (3,4), (4,5), (0,3), (1,4), (2,5)\}$, then per-qubit Rot), and it measures only Z , X , and ZZ (19 observables: $6+6+7$) — no XX/YY terms. This means only **two** measurement circuits per patch are needed (all- Z , all- X). For four CIFAR images at 16 patches each, this is $4 \times 16 \times 2 = 128$ circuits.

4 Local Validation Before Any Hardware Submission

Every circuit was validated against the cached, training-time observable vectors (`observables.npy`, produced by the original PennyLane run) via an exact Qiskit statevector simulation (`Statevector.from_instruction` *before* any job was submitted to real hardware. Maximum absolute error per patch, across all measured observables:

Table 1: Local validation: Qiskit statevector reconstruction vs. cached PennyLane training-time observables.

Checkpoint	Patches validated	Max abs. error
Monalisa (HVK1D)	16	7.89×10^{-4}
CIFAR cat (HVK2D)	16	9.42×10^{-6}
CIFAR ship, hydrofoil (HVK2D)	16	7.76×10^{-6}
CIFAR ship, sea boat (HVK2D)	16	7.89×10^{-6}
CIFAR airplane (HVK2D)	16	7.14×10^{-6}

The Monalisa error (7.89×10^{-4}) is an order of magnitude larger than the CIFAR errors ($\sim 10^{-5}$ – 10^{-6}); both are far below anything that would explain the hardware-vs-simulator PSNR gap reported in the main text, so the gap in the real-hardware results is attributable to genuine device noise, not a circuit-translation error. A dry-run mode in the pilot scripts (`IBM.Cloud/run_hvk_hardware_reconstruction` and `IBM.Cloud/run_hvk2d_cifar_hardware_reconstruction.py`) reruns this check and reports simulator-only decode PSNR without touching the network or any account credentials, so the validation step costs nothing and requires no IBM account.

5 IBM Quantum Account and Backend

Jobs were submitted via `qiskit-ibm-runtime` (channel `ibm_quantum_platform`) under a free/open-plan instance. Backend selection preferred the operational 6+-qubit backend with the fewest pending jobs at submission time; both pilots ran on `ibm_fez`, a 156-qubit Heron-class processor, which had zero queued jobs ahead of ours at submission time for the Monalisa job and remained uncongested for the CIFAR job. Transpilation used `optimization_level=1`. Each job used `SamplerV2` with 256 shots per circuit.

6 Job Identifiers and Quota Accounting

Table 2: IBM Quantum job identifiers for the hardware pilot (backend: `ibm_fez`, 256 shots/circuit).

Image	Job ID	Circuits	Bases
Monalisa	d9ecu34inv1c73aq3qt0	48	Z, X, Y
CIFAR cat	d9edfo2neu4c739ob2ig	32	Z, X
CIFAR ship (hydrofoil)	d9edg5cjeosc73filhng	32	Z, X
CIFAR ship (sea boat)	d9edgakjeosc73filhug	32	Z, X
CIFAR airplane	d9edgdphtsac739e4kdg	32	Z, X

Total: 176 circuits across 5 jobs. Quota consumption, read directly from the account’s `QiskitRuntimeService.usa` endpoint immediately before and after the pilot: $19 \rightarrow 35$ seconds consumed against a 600-second monthly free-plan allowance — i.e., the entire five-image, 176-circuit pilot cost 16 seconds of billed quantum-processor time, with 565 seconds remaining afterward. This is reported so future extensions of this pilot (more images, more shots, or the full held-out suite) can be budgeted realistically: at the observed rate ($\approx 0.09\text{--}0.13$ s/circuit), even a substantially larger pilot remains inexpensive on a free-plan account.

7 Reconstruction Decode

Hardware counts were converted to observable expectation values by the standard parity estimator, $\langle Z_i \rangle = \frac{1}{S} \sum_{\text{shots}} (-1)^{b_i}$ (and identically for X , Y after basis rotation), with pairwise correlators $\langle P_i P_j \rangle$ computed as the shot-weighted mean of $(-1)^{b_i + b_j}$ within the same measurement basis — no additional circuits were needed for the pairwise terms. The resulting 27-D (HVK1D) or 19-D (HVK2D) hardware observable vector, concatenated with the same sinusoidal positional encoding used at training time, was passed through the *unmodified* trained decoder to produce the hardware reconstruction reported in the main text. No hardware-specific fine-tuning, error mitigation, or post-hoc calibration was applied; the PSNR gap between simulator and hardware in the main text is therefore a raw, unmitigated noise signature, and a natural target for future error-mitigation follow-up work.

8 Reproducibility

Both pilot scripts default to a dry run (simulator-only validation, no network calls, no credentials) and require an explicit `--submit` flag plus a circuit-count safety check (raising an error above 20

circuits unless `--allow-large-job` is passed) before touching real hardware, so the validation and the hardware-submission steps cannot be conflated by accident:

- `IBM_Cloud/run_hvk_hardware_reconstruction.py` — Monalisa/HVK1D pilot.
- `IBM_Cloud/run_hvk2d_cifar_hardware_reconstruction.py` — CIFAR-10/HVK2D pilot.
- `Baselines/cifar10_comparisons/hvk2d/train_and_save_hvk2d_checkpoints.py` — trains and saves the four CIFAR checkpoints reused above.